

Licence 3 Informatique

Année Universitaire 2025 – 2026

Implémentation d'algorithmes de réduction

Période du stage :

du 13 avril au 28 août 2026

Rédigé par :

Stella-Rose MILLE

Tuteur Universitaire :

Romain ROUVOY

Tuteur en Entreprise :

Guillermo POLITO

Superviseur :

Federico LOCHBAUM

Table des matières

1	Contexte du Stage	2
1.1	Présentation du Laboratoire et de l'Équipe	2
1.2	Contexte de l'Application	2
2	Description d'une réalisation technique majeure	3
2.1	Contexte et objectif du shrinking	3
2.2	Choix techniques et justifications	3
2.3	Développements et travaux réalisés	4
2.4	Difficultés rencontrées et solutions apportées	5
3	Conclusion et Bilan du Stage	6
3.1	Adéquation de la Réalisation aux Objectifs et Perspectives	6
3.2	Présentation du Travail à Réaliser pour la Suite	6
3.3	Enseignements et Apports du Stage	6
3.4	Remerciements	6

1. Contexte du Stage

1.1 Présentation du Laboratoire et de l'Équipe

Ce stage se déroule au sein d'INRIA (Institut National de Recherche en Sciences et Technologies du Numérique), un établissement de recherche dédié aux sciences du numérique. J'y ai intégré l'équipe de recherche Evref, une équipe dont le principal objectif est l'étude de l'évolution des grands systèmes informatiques. Leurs travaux s'articulent autour de trois axes : L'analyse et la migration de système existants, le développement d'outils pour supporter l'évolution et l'infrastructure permettant de supporter l'évolution logicielle.

1.2 Contexte de l'Application

Mon travail s'inscrit dans le cadre d'un projet de thèse de l'équipe, centré sur le développement d'Ume : un framework de Property-Based Testing (PBT) conçu spécifiquement pour l'écosystème Pharo (un langage purement orienté objet développé par l'équipe). Contrairement aux tests unitaires classiques où le développeur écrit manuellement les inputs, le PBT génère automatiquement des centaines d'inputs pour valider le bon fonctionnement du logiciel. Ume cherche à spécialiser ce concept dans la recherche de problèmes de performances.

Au sein d'Ume, on organise cette génération de tests autour d'une campagne de tests : une génération automatique d'un large nombre de tests définis par un schéma.

Le Schema est un objet définissant précisément la forme des données à générer et la condition de validation. Elle se repose sur quatre composants : le générateurs (qui décrit la manière dont l'objet doit être initialisé), la contrainte des arguments (un tableau contenant des générateurs de contraintes, chacun correspondant à un argument attendu), la méthode (que l'on veut tester), la propriété (vérifiant si le cas généré est fonctionnel). Une campagne a un cas d'arrêt défini par l'utilisateur, cela peut être une contrainte de temps, de nombre de tests générés, d'erreurs rencontrés ou la couverture de code.

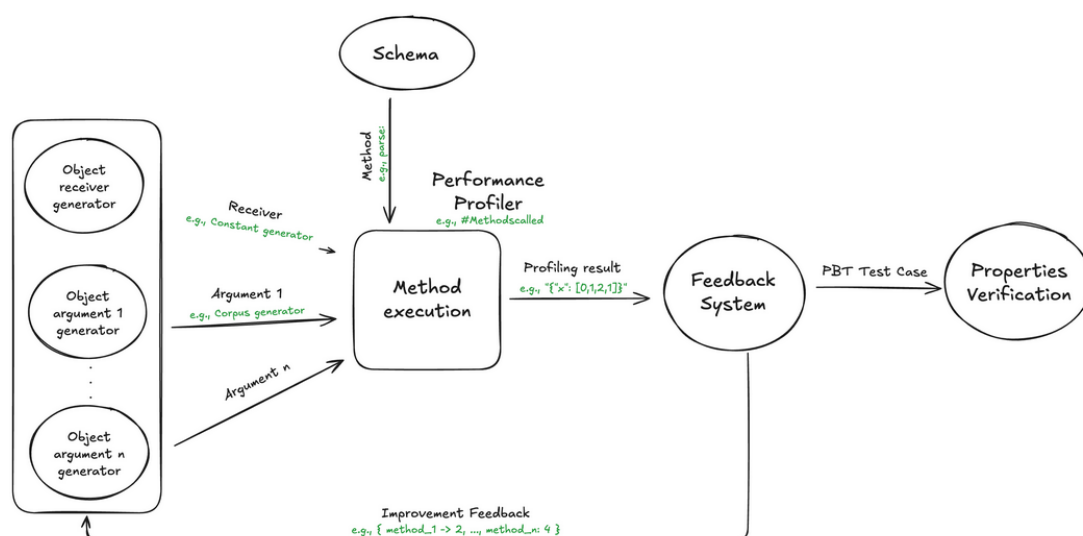


FIGURE 1.1 – Architecture d'Ume ¹

1. <https://github.com/FedeLoch/Ume>

2. Description d'une réalisation technique majeure

Pour cette section, j'ai décidé de présenter un axe de mon stage consistant en l'implémentation du shrinking au sein d'Ume.

2.1 Contexte et objectif du shrinking

Le shrinking consiste en la réduction d'entrée de tests : lorsqu'un test échoue sur un input complexe générée durant une campagne de fuzzing¹. L'objectif est de réduire cet input à une taille minimale reproduisant la même erreur. Bien que l'utilité de l'application du shrinking à des problèmes de performances est une question que l'on se pose, cette fonctionnalité reste indispensable d'un point de vue utilisateur pour permettre la localisation précise du problème et simplifier la phase de debuggage.

2.2 Choix techniques et justifications

Afin d'intégrer un mécanisme de shrinking dans Ume, je me suis basé sur les algorithmes présentés dans le Fuzzing Book².

Pour débiter, j'ai implémenté un algorithme de Delta Debugging classique qui permet de traiter la réduction de chaînes. Cet algorithme est basé sur le principe du "diviser pour régner", il segmente les chaîne de caractères et évalue chaque morceau afin d'isoler la faille. (figure 2.1)

Dans l'exemple suivant, le problème d'une chaîne réside dans la présence du caractère 'e'. Ainsi la chaîne 'kqsbdvieqvsbdo' va être d'abord fragmentée en deux : on évalue le côté droit, si il ne retourne pas d'erreur on évalue le côté gauche. Si un morceau de la chaîne reproduit l'erreur, on peut le fragmenter à nouveau. La réduction s'arrête dans la chaîne atteint la taille minimale de un caractères ou lorsque on ne peut plus segmenté en provoquant une erreur. Ainsi notre chaîne 'kqsbdvieqvsbdo' est réduite à quatre reprises et on obtient 'e'.

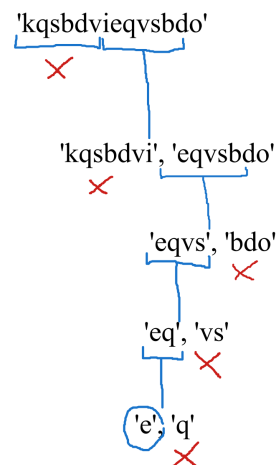


FIGURE 2.1 – exemple de shrinking sur une chaîne

1. dire ce qu'est le fuzzing

2. <https://www.fuzzingbook.org/html/Reducer.html#Delta-Debugging>

Malgré son efficacité pour traiter des chaînes, le Delta Debugging montre ses limites au moment de traiter des grammaires complexes. Une simple segmentation casse la syntaxe et les tests échoue pour des erreurs de parsing.

C'est pourquoi il est nécessaire de coupler cette méthode à des réducteurs basés sur la grammaire, afin de garantir que chaque entrée soit syntaxiquement valide. Une entrée utilisant une grammaire spécifique est représenté sous la forme d'un arbre :

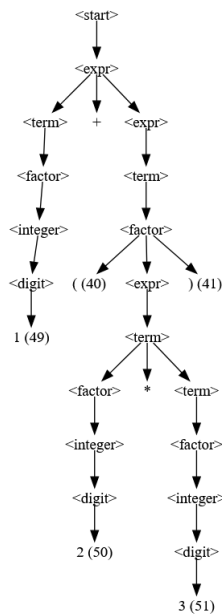


FIGURE 2.2 – arbre de grammaire

On remarque la présence de règles de grammaires, on ne peut pas segmenter l'arbre de manière aléatoire, il est nécessaire de le faire en respectant la grammaire utilisé par le développeur.

2.3 Développements et travaux réalisés

Les algorithmes restant complexes, j'ai appliqué la méthodologie du test-driven development (TDD). Cela m'a permis de définir le comportement attendu avant même de commencer le code. J'ai implémenté deux algorithmes de réduction avec un design pattern Strategy : la réduction par sous arbres et par expansion alternée. Cette méthodologie permettras d'implémenter facilement l'ajout de futurs algorithmes.

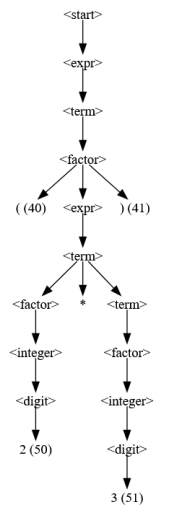


FIGURE 2.3 – Stratégie de réduction par sous-arbres

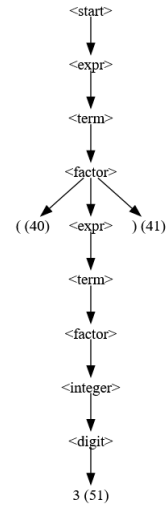


FIGURE 2.4 – Stratégie par expansion alternée

La réduction par sous arbres : Afin de réduire en utilisant les sous arbres, on choisit un noeud de notre arbre et on le remplace par un sous-arbre ayant le même noeud. Dans la figure 2.3, le premier noeud `<expr>` est remplacé par un sous arbre `<expr>` correspondant à $(2*3)$.

La réduction par expansion alternée : On peut réduire d’une autre manière en regardant si il existe des sous arbres d’un même symbole avec moins d’enfants. Dans la figure 2.4, on trouve une alternative au noeud `<term>` à 2 enfants par le sous arbre `<term>` avec un seul enfant correspondant à 3.

2.4 Difficultés rencontrées et solutions apportées

En implémentant le shrinking, j’ai rencontrés plusieurs difficultés. Les exemples d’algorithmes du Fuzzing Book sont écrits en Python, un langage bien plus permissif que Pharo. J’ai notamment rencontrés des problèmes lors de la manipulation d’arbres, dans le cas de modification ou de récupération d’informations sur des noeuds. J’ai réglé ce problème en analysant directement l’implémentation des arbres dans Pharo afin de prendre connaissances des méthodes que j’avais à disposition.

Avec le Fuzzing Book sous la main, je me suis perdue sur ce que je devait faire et dans quel ordre le faire. Afin d’y remédier, Federico m’a proposé de faire du TDD, en commençant par des tests simple et de graduellement complexifier mes entrées.

3. Conclusion et Bilan du Stage

3.1 Adéquation de la Réalisation aux Objectifs et Perspectives

Au cours des dernières semaine, j’ai réalisé :

- La simplification de la définition d’une campagne
- l’ajout d’une barre de progression
- création d’une CI (intégration continue) limité à certains tests
- implémentation du shrinker : delta debugger et réductions de grammaire

L’ajout de ses fonctionnalités permet une prise en main plus simple pour un utilisateur : la définition est concise et a un retour visuel prouvant que des tests sont générés en temps réel. Le shrinker peut être utilisé afin de trouver des cas d’erreurs debuggables facilement, et même avec l’utilisation d’une grammaire spécifique.

Concernant la question de l’évaluation, la mise en place de benchmarks constitue l’étape logique suivante. Ils permettront de comparer différents algorithmes de shrinking.

3.2 Présentation du Travail à Réaliser pour la Suite

Mon stage s’étendant jusque fin août, j’ai plusieurs missions pour la suite, le shrinker doit continuer à être développé, notamment par l’implémentation d’autres algorithmes de réductions et par la comparaison entre eux. Afin d’avoir une lecture plus simple des résultats d’une campagne, l’ajout d’un feedback visuel est prévue. Ce retour permettra d’avoir une vue d’ensemble sur la campagne et d’en recueillir les informations facilement. Et dans cette continuité, l’ajout d’un plugin pour Dr Test (une UI plugin-based permettant de gérer les tests unitaires) est aussi prévue.

3.3 Enseignements et Apports du Stage

Ce stage m’a permis de développer une forte autonomie. Dès les premiers jours, mon tuteur a mis l’accent sur l’organisation et l’autonomie. Nous avons structuré notre flux de travail avec GitHub pour la gestion de versions (apprentissage des Pull Requests bien documentées) et Obsidian pour la documentation de mes tâches quotidiennes.

Sur le plan technique, l’utilisation rigoureuse du TDD m’a permis de me canaliser lorsque je perdais de vue l’objectif final, tandis que l’application concrète des Design Patterns a renforcé mes compétences en conception logicielle.

3.4 Remerciements

Je souhaite remercier mon tuteur d’entreprise, Federico Lochbaum, pour ses précieux conseils, sa patience et sa confiance. À Guillermo Polito, pour son accompagnement et pour s’être toujours montré disponible pour me renseigner. Mon tuteur universitaire, Romain Rouvoy, pour son suivi rigoureux. Enfin, j’adresse mes remerciements à Imen Sayar pour son soutien continu et pour avoir appuyé ma candidature pour ce stage.